HOSTING AND REALIZABILITY IN THE SELECTED PR589 BIT SUPPLIER

DAN BROWN* AND KENNY DANIEL

Abstract. The selected bit supplier of the community bound on integer multiplication [1], PR589 on PR555, prices one interchange node at the saving $a_b = 10/5769$. Its accounting makes every rank-r frame change of its word one child interchange of width rb, and its routes and charts free. We record what happens when this node is ported to the recursive interchange machine that is formalized in Lean [5]. (A) In one invocation a family hosts up to five helpers. Upstream separates them by classes of an orthogonal-group cover. A machine with a fixed role count can instead split each family into five row classes, which leaves the occurrence moment unchanged. The role-level node theorem of this scheme is checked in Lean. (B) The machine's cheap operations are lower triangular, so a frame change whose projector difference is not diagonal is not one child, and the routes and charts are coordinate changes that cost interchanges. Oblique flag frames, with each frame change split into runs of consecutive pivots, turn the frame changes into cheap conjugates of children. The routes remain: in a stated machine model, which charges sub-field transpositions as children of the same recursion, they raise the occurrence moment of upstream's compressed geometry above one, and no saving remains. The saving needs the compression, since an uncompressed block geometry loses its only free coordinate. These are arguments on paper with computation, not Lean theorems. The no-go holds for architectures without class-indexed covers. Upstream's later packages pay for the routes with a cover indexed by $\mathrm{GL}_{64}(\mathbb{Z}/q^w)$, a different machine whose completion is open by upstream's own account. The finite certificates of the supplier, re-checked in Lean, agree with upstream.

1. Introduction

The verified multiplier of [5] runs in time $O\big(n(\log n)^{1-\kappa}\big)$ on Mathlib's finite multi-stack Turing machines, with $\kappa = 88279 \cdot 2^{-46}$. Upstream's selected record [1] combines a complex supplier (T16) with a bit supplier (PR589, tuned by DreamingOfClouds on eumemic's PR555) and states the conditional exponent $\kappa = 0.001672660197076918$. Its README lists the open parts: "All-size/compiler, common-label/mutable-source realization, whole-call hosting, precision/tape, exact recovery and analytic obligations remain conditional."

The Lean development runs T16 on its complex side, through Kenny Daniel's T16 node on the layer machine, and its bit interchange binds κ. Under upstream's accounting, adding PR589 would move the cap on κ from about $2^{-29.4}$ to about $2^{-10.3}$. This note records what the port of PR589 found.

Each claim carries a tag. [Lean] marks a theorem checked in [5], under the name given, with only Lean's standard axioms. [paper] marks a written argument, here or in the design notes of the development, that is not formalized. [computation] marks a number computed by a script from upstream's pinned data. Upstream sources are pinned at the scientific base `0fdd7d82` and, for later packages, at `463113d6`.

2. The supplier and its bill

The parent of a bit interchange node holds records whose addresses are pairs (x, y), with x the H field and y the D field, each of 64 coordinates; the interchange exchanges them. For an idempotent U write

$$D_U(x, y) = \big((1 - U)x + Uy,\ Ux + (1 - U)y\big),$$

so the parent interchange is D_1 (`exchange` in `Nine/PortFrame`). The supplier's word has 5583 instructions on 661 registers. Each register carries a frame: a label $V \subseteq \mathbb{Q}^8$ spanned by E_8 lines, with

Date: October 18, 2026.

*Author of *The Da Vinci Code*.

orthogonal projector P_V. A MOVE takes a register from P_V to $P_{V'}$ with $V \subseteq V'$, and `exchange_change` [LEAN] gives $D_{P_V} D_{P_{V'}} = D_{P_{V'} - P_V}$. Its rank is $\operatorname{rank}(P_{V'} - P_V)$.

The node runs the word in nine stages over four pairs of data banks, in 32 replicas, on 111 live ports, with 439 helpers per invocation hosted in idle families. It has $W = 28416$ live roles. The accounting makes every rank-r MOVE of every invocation, and every hosted idle remainder of rank r, one child interchange of width rb, where $b = w/64$ for a parent of width w. The children of one node then have the rank histogram

rank	1	2	3	4	5	6	7	42
count	381024	400416	22752	26880	29376	17376	26784	288

which is `bitHistogram`, equal to upstream's profile. [LEAN]

- `nodeChildren_realizes`: the node's children, in schedule order, realize `bitHistogram`.
- `bit_bill_le`: the occurrence moment $\frac{1}{W} \sum_{\text{children}} (r/64)^{1-a_b}$ plus its surcharge is at most 1, at $a_b = 10/5769 \approx 1.7334 \cdot 10^{-3}$.
- `bit_variable_closed`: a node whose children have sizes at most $r/64$ of the parent (the premise `hsize`) and weight $1/W$ costs $O(e^{\lambda})$ for every $\lambda > 1 - a_b$.

These results take the accounting as given. Its two assumptions are the subject of this note: a rank-r frame change is one child of width rb, and routes and charts are free.

3. Hole A: hosting

3.1. **Five helpers in one family.** In one invocation a family of an inactive pair can host up to five helpers. They lie on slots of a single idle (`same_pair_windows` [LEAN]) at offsets $8j$ with $j < 5$ (`slot_fits` [LEAN]). Upstream's hosting audit [4] counts the same maximum of five. In upstream's first example [2], at stage 2, replica 0, pair 0, helpers 232 through 236 share family 666, on the supports 1..8, 9..16, 17..24, 25..32 and 33..40 of idle child 4's 42-dimensional pending projector. The helpers are separate registers with possibly correlated contents, so they need separate storage.

3.2. **Upstream's cover.** Upstream separates them with a cover [2, §§2,4]. A physical stream is indexed by a pair (F, g), with F a family and g an element of the full orthogonal group G of the form $I_{64}/2$ over a finite field, and a cover sweep visits every element of G once. At invocation d the helper on slot S is owned by $(F, d\sigma_S^{-1})$, so helpers that share F have different classes. Upstream's later row allocations carry this group in the row stock, through the factor $|\mathrm{GL}_{64}(\mathbb{F}_q)|\, q^{4096(w-1)}$ [6]. The machine of [5] has a fixed role count, and its row divisor is a power of that count (`RecursiveRowDivisor`). It has no cover classes.

3.3. **Five row classes.** Split each family's rows into five classes. Run each invocation once per class $d \in \mathbb{Z}/5$, and give the helper on the slot at offset $8j$ the class $(d + j) \bmod 5$ of its host family (`hostClass`).

- [LEAN] `hostClass_distinct`: two distinct slots of one stage, pair and copy receive distinct classes for every d. `hostClass_bijective`: as d runs through $\mathbb{Z}/5$, each slot visits each class of its family once. `card_role`: the node has $5W = 142080$ roles.
- [LEAN] One class run, on these roles: both sides of the active pair change between their stage frames, and a hosted helper's role changes by the exchange on its slot (`invoke_side_zero`, `invoke_side_one`, `invoke_host`). All 160 class runs of a stage: two runs host orthogonal slots on a role (`hostSpan_orthogonal`), each residual of the stage arrives on its role exactly once (`hostSpan_sum`), and each role of another pair changes by the exchange on the sum of the slots its family hosts at that stage (`stagePiece_inactive`).
- [LEAN] The node: `nodeProgram` has 29 pieces, and on arbitrary dirty streams it changes every role by D_1 (`runNode_interchange`). Every child appears five times, with weight $1/(5W)$: the child list realizes five times `bitHistogram`, and the occurrence moment $\frac{1}{5W} \cdot 5 \sum (r/64)^{\lambda}$ is at

most 1 at $\lambda = 1 - a_b$ (`five_class_node`). `five_variable_closed` is the closed recurrence at weight $1/(5W)$, with the per-node cost as a hypothesis.

These semantics are at role level. Streams are functions of rational address pairs; routes, charts and frame changes act on them as exact operators; and each frame change is priced as one child of its rank. The scheme is sound at that level. Its realization on tape depends on hole B.

4. Hole B: realizability

4.1. **The machine model.** The arguments of this section use the following model of a node of the recursive interchange machine, on volume V, with each address field split into sub-fields of b digits stored in H-before-D order.

(1) The nonrecursive work of a node costs $O(V)$, with a constant independent of the width.
(2) Linear passes are: scalings; shifts whose control sub-field is more significant than the target in the H-before-D layout; cross shears $D \mathrel{+}= MH$; XORs and copies of role arrays; and role relabeling. On the address coordinates of one role these act by lower triangular matrices. In [5] the scalings and shifts are `FlatCoordinateSchedule.Op`, whose shift control precedes its target. The shear programs of the manuscript [7] have the same form: Lemma 4.2, `exists_prime_shear` [Lean], uses exactly rank A interchanges and otherwise lower triangular transformations, and every transformation of the generated Shared50 program is lower triangular (`programs_triangular` [Lean]).
(3) Moving data from a less significant into a more significant sub-field, as in upper transvections and sub-field transpositions, is charged as an interchange child, since a transposition of two sub-fields of width b is itself an interchange instance. A child of rank r swaps r consecutive H sub-fields with r consecutive D sub-fields, at width rb (`childR` [Lean]).
(4) Upstream's cover model sidesteps (3), and upstream lists the all-size compiler as an inherited assumption.

4.2. **Frame changes.**

Proposition 4.1 ([paper])**.** *Let A and B be lower triangular in the H-before-D order, and let E be a diagonal $0/1$ matrix. The upper-right block of AD_EB is lower triangular. Hence, if U is symmetric and not diagonal, D_U is not of the form AD_EB.*

Proof. Write $A = \begin{pmatrix} A_1 & 0 \\ A_3 & A_4 \end{pmatrix}$ and $B = \begin{pmatrix} B_1 & 0 \\ B_3 & B_4 \end{pmatrix}$, with A_1, A_4, B_1, B_4 lower triangular. Since $D_E = \begin{pmatrix} 1-E & E \\ E & 1-E \end{pmatrix}$, the upper-right block of AD_EB is A_1EB_4, a product of lower triangular matrices. The upper-right block of D_U is U, and a symmetric lower triangular matrix is diagonal. □

So with orthogonal frames, a MOVE whose projector difference $P_{V'} - P_V$ is not diagonal is not one child between linear passes. In the word's local coordinates, 2787 of its 2911 paid MOVEs (phase incidences of positive rank) have a non-diagonal difference [computation].

[paper] Oblique flag frames remove this obstacle. Give each label V the idempotent Π_V with range V whose kernel is spanned by the coordinate vectors outside the pivot set S_V of V, the set of leading positions of nonzero vectors of V. From a reduced echelon basis, Π_V is lower triangular with diagonal 1_{S_V}. For $V \subseteq V'$ one has $S_V \subseteq S_{V'}$ and $\Pi_V\Pi_{V'} = \Pi_{V'}\Pi_V = \Pi_V$, so `exchange_change` applies with these frames, and the frame change is D_U with $U = \Pi_{V'} - \Pi_V$, of rank $\dim V' - \dim V$. The idempotent U is lower triangular with diagonal $E = 1_{S_{V'} \setminus S_V}$, so $U = CEC^{-1}$ for a lower triangular C, and $D_U = \operatorname{diag}(C, C)\, D_E\, \operatorname{diag}(C, C)^{-1}$ (`exchange_conj` [Lean]). The frame change is a cheap conjugate of the swap of its new pivot coordinates. The Lean frame identities of §6 use orthogonal frames; their oblique counterparts are not formalized.

[COMPUTATION] The new pivots need not be consecutive, and `childR` needs consecutive sub-fields. Over the 40320 orders of the local axes, the order $(3, 0, 2, 4, 6, 5, 7, 1)$ gives consecutive new pivots for 2281 of the 2911 paid MOVEs, which carry 3458 of the rank mass 5066 (68%). Splitting each MOVE into one child per maximal run of consecutive new pivots gives the run histogram {1:2443, 2:979, 3:86, 4:35, 5:35, 6:6, 7:8}, 3592 runs. Each run is a cheap conjugate of a `childR` child, since the exchanges on orthogonal diagonal projectors multiply (`exchange_mul_of_orthogonal` [LEAN]).

4.3. **Routes in the compressed geometry.** The stage route of a port with line q is $\rho = \begin{pmatrix} P & A \\ A^{\mathsf{T}} & 0 \end{pmatrix}$ on the eight local axes and one exterior block of seven, with $P = qq^{\mathsf{T}}/4$. It is a symmetric involution (`route_transpose`, `route_mul_self` [LEAN]), with entries in $\{0, \pm\frac{1}{4}, \pm\frac{1}{2}, \pm 1\}$, and a lifted MOVE is the route conjugate of the local one, extended by zero (`lift_sub` [LEAN]). The slot charts embed a helper's eight local axes isometrically onto its slot (`slotChart_orthonormal` [LEAN]). Routes and charts are orthogonal and not triangular. In upstream's cover model they are free: $g\sigma = d$ for every role, so "a chart-routed ADD is exactly a common native address ADD" [2, §2]. On a tape they are coordinate changes, that is, permutations of the address space applied to stored records.

In the model of §4.1, with route transpositions executed as children of this recursion, the routes cost more than the saving.

- [COMPUTATION] Every invertible matrix factors as $L_1 \pi L_2$ with L_1, L_2 lower triangular and π a unique permutation (Bruhat). With the local axes in the order $(3, 0, 2, 4, 6, 5, 7, 1)$ and the exterior block after them, the permutation of each of the 111 port routes has 7 transpositions on each field.
- [PAPER] The routes must depend on the port: a route common to all ports would map all 111 lines to one axis.
- [PAPER] By (3), executing a route through its Bruhat form costs sub-field transpositions, each an interchange child. The feasibility study counts about 4.5 rank-seven interchanges, of width $7b$, per role per node.
- [PAPER] At $\lambda = 1$ the occurrence moment is the rank mass over $64W$. Without routes the mass per role is $1808352/W \approx 63.6385$, so the moment is $63.6385/64 \approx 0.99435$, a slack of about 0.0056. The routes add about $4.5\,(7/64)^{\lambda} \approx 0.49$ to the moment near $\lambda = 1$, so it exceeds 1 for every $\lambda \leq 1$. No saving remains.
- [COMPUTATION] Without routes, splitting each MOVE into consecutive runs would give the saving $1.6407 \cdot 10^{-3}$, and rank-one children $1.362 \cdot 10^{-3}$. Both still need the routes of the compressed geometry, so neither escapes the count above.

4.4. **Why the saving needs the compression.** [PAPER] A bit saving needs rank mass below mW at $\lambda = 1$, for parent width m. The only free coordinate of PR589 is the shared port line. It is worth 1 per data column, against a COPY overhead of $L \cdot \mathrm{CST}/(2KV) = 9 \cdot 63/888 \approx 0.638$, where $\mathrm{CST} = 63$ is the sum of the nine COPY ranks. That leaves about 0.362 per role, and $W \cdot 0.362 \approx 10272$ is upstream's deficit $T(2KV - L \cdot \mathrm{CST})$ at $T = 32$, $K = 4$, $V = 111$, $L = 9$ [11].

In an uncompressed block geometry, with each stage running the word on its own block of eight axes and parent width $m = 72$, no routes or charts are needed. Without compression, the asymmetry between the two sides leaves the stage window after the first stage. Moving the line to a port-independent axis u makes the bank matrix a scalar multiple of the identity, so the free coordinate cancels [COMPUTATION]. The mass per role is then $72.638 > 72$, and the moment at $\lambda = 1$ is $1.0089 > 1$. The uncompressed geometry has no saving.

4.5. **Upstream's later treatment of the routes.** After the base, upstream merged packages that price the routes explicitly, at `463113d6`. `selected-bit-stream-routes` [10] compiles the 999 data and 124 hosted route types into exact rational column words and charges up to 3520 ordinary interchange calls per route action. `selected-bit-recursive-caller` [8] states that "its routing and leaf swaps still

cost $O(wV)$". `selected-bit-fast-primitives` [3] replaces both ordinary-interchange sites with a previously constructed native interchange and claims a local toll $O_q\big(V(n+w^{1-a_1})\big)$ with $a_1 = 169711/10^8$. `selected-bit-recursive-closure` [9] claims ordinary binary chunk interchange in $O(VE^{1-a})$ with $a = 0.00169711$, in its fixed-tape complete-stream model. Their authors present them as written arguments without independent or proof-assistant verification.

[PAPER] Their toll rests on a class-indexed cover. Every stream is indexed by $g \in \mathrm{GL}_{64}(\mathbb{Z}/q^w)$, written $g = g_0 + qG$ with a low class $g_0 \in \mathrm{GL}_{64}(\mathbb{F}_q)$ and 4096 high fields G_{ij} of $w-1$ digits, and each packet is ordered as its low-class label, then the high fields, then the payload [10, PROOF.md §1]. A route is the right action $g \mapsto g\sigma$ on this label of fixed size: a block permutation of the low classes, plus carry-aware updates of the high fields through the previous-stage interchange at width $w-1$ ("B1, ordered ADD, B1" [3, PROOF.md §4]). A route therefore costs $O_q(Vw^p)$ for a fixed exponent p, independent of the payload width, because it never moves payload address digits.

[PAPER] In the model of §4.1 roles carry no class label, so a route acts on the payload's own 64 sub-fields of $n/64$ atoms each. Whether it runs as q-adic sub-field arithmetic or as atom-wise elementary operations, its cost per node is about Vnw^p: a product of the two widths where upstream's toll has a sum, superlinear in the payload width.

[PAPER] The cover costs volume. The stock carries a factor $|\mathrm{GL}_{64}(\mathbb{Z}/q^w)|$, constant for fixed q and w, with $|\mathrm{GL}_{64}(\mathbb{F}_q)| > 2^{327680}$ at $q > 2^{80}$, and children need their own classes, through the stock and a fresh-class contract. The saving also needs w to grow: at fixed w the time per unit volume is $\Theta(n)$. It comes from balancing $n \approx e^{\theta}$ against $w \approx e^{1-\theta}$, which gives $a_{k+1} = c(1-c) + c\,a_k$ with $c = 17/10000$ [3].

[COMPUTATION] If that architecture were completed, one bootstrap from the Shared50 interchange would give $a_b \approx c(1-c) \approx 1.697 \cdot 10^{-3}$, and with T16, $\kappa \approx 2^{-10.82}$ with rank-one T16 children or $\kappa \approx 2^{-10.44}$ with variable-rank ones.

So the no-go of §4.3 holds for architectures without class-indexed covers. Upstream's cover architecture is a different machine, and its completion is open by its own account: "We retain the data-dependent recursive compiler, weighted calls, correlated-helper domain bridge and COPY lifetime obligations. [...] The selected result remains conditional." [10]

5. Paths forward

Besides completing upstream's cover architecture (§4.5), three paths remain.

(i) A bit word whose sources share a port-independent line, so that compression needs no routes.
(ii) A primitive that makes the needed sub-field moves linear, against which assumption (3) of §4.1 argues.
(iii) A different bit supplier.

The hosting scheme of hole A and the oblique frames of §4.2 apply to any compressed geometry with these hosting windows, so they carry over to path (i).

6. What holds

The finite certificates of the supplier are re-checked in Lean and agree with upstream. [LEAN]

- Local MOVE profile: every phase incidence is a nested frame pair whose rank is the certified dimension difference (`phase_ranks`, `phase_histogram`); final alignment pays no rank (`final_ranks`); the nine COPY ranks are all 7 (`copyRanks_eq`); the local paid histogram is upstream's {1:1323, 2:1342, 3:79, 4:44, 5:102, 6:11, 7:19} (`localPaid_histogram`).
- Stage geometry: the 111 port certificates repeat upstream's reflection walk (`ports_checked`); each column leaves a stage in the frame in which it enters the next (`stage_source`); after nine stages and the exchange, every column's frame is D_1 after its initial frame (`node_exchange_source`); lifts are route conjugates (`lift_sub`).

- Windows: upstream's nine-stage windows (`windows_four`), and every hosted stage lies in its idle's window (`selection_in_windows`).
- Occurrence link and bill: `nodeChildren_realizes`, `bit_bill_le`, and the recurrence `bit_variable_closed`.

The occurrence link and the recurrence stand, given the accounting. Hole B asks whether a machine realizes the accounting.

The complex side has no analogue of either hole with rank-one children. Kenny Daniel's T16 node program runs on a fixed set of 14336 data families and eight private centres, with no cover classes. Every family ends with C_{all} applied to its input (`nodeProgram_cAll` [LEAN]), its child calls are 1041408 rank-one odd kernels (`nodeProgram_children` [LEAN]), and in every column such a kernel is the layer machine's factor (`colRep_kernel` [LEAN]).

7. STATUS

item	status
Profile, stage geometry, windows, occurrence link, bill, recurrence	Lean
Five classes: class facts, one class run, all class runs of a stage	Lean
Five classes: node program, node theorem, moment at weight $1/(5W)$	Lean
A MOVE with non-diagonal difference is not one child (Prop. 4.1)	paper
2787 of 2911 paid MOVEs have a non-diagonal difference	computation
Oblique flag frames make frame changes cheap conjugates of children	paper
Pivot runs at the best axis order	computation
Bruhat counts of the 111 routes	computation
Compressed geometry: no saving in the model of §4.1	paper and computation
Uncompressed block geometry at $m = 72$: no saving	paper and computation
Cheap routes need a class-indexed cover (§4.5)	paper
Upstream's cover architecture	open by upstream's account

REFERENCES

[1] D. Colkitt (maintainer) and contributors, `integer-mult-bounds`: community research on integer multiplication below $n \log n$, base `0fdd7d82`, head `463113d6`, https://github.com/CrocSwap/integer-mult-bounds.

[2] `research/pr589-five-helper-hosting`, PROOF.md, in [1] at `0fdd7d82`, https://github.com/CrocSwap/integer-mult-bounds/tree/0fdd7d82de03e43c9e5a1000dfbcd380d1b687d2/research/pr589-five-helper-hosting.

[3] `research/selected-bit-fast-primitives`, in [1] at `463113d6`, https://github.com/CrocSwap/integer-mult-bounds/tree/463113d675af7410ac91d53a4fed51ec6494eb9a/research/selected-bit-fast-primitives.

[4] `research/selected-bit-hosting-audit`, in [1] at `0fdd7d82`, https://github.com/CrocSwap/integer-mult-bounds/tree/0fdd7d82de03e43c9e5a1000dfbcd380d1b687d2/research/selected-bit-hosting-audit.

[5] K. Daniel, `integer-mult-bounds-lean`: integer multiplication below $n \log n$ in Lean on Mathlib's finite Turing machines, https://github.com/platypii/integer-mult-bounds-lean.

[6] `research/selected-bit-local-ring-sweeps`, in [1] at `463113d6`, https://github.com/CrocSwap/integer-mult-bounds/tree/463113d675af7410ac91d53a4fed51ec6494eb9a/research/selected-bit-local-ring-sweeps.

[7] OpenAI, *Integer multiplication below $n \log n$*, OpenAI Math Release preprint, September 23, 2026, https://github.com/openai/math/blob/main/preprints/Integer-multiplication-below-n-log-n-September-23-2026/paper.pdf.

[8] `research/selected-bit-recursive-caller`, in [1] at `463113d6`, https://github.com/CrocSwap/integer-mult-bounds/tree/463113d675af7410ac91d53a4fed51ec6494eb9a/research/selected-bit-recursive-caller.

[9] research/selected-bit-recursive-closure, in [1] at 463113d6,
https://github.com/CrocSwap/integer-mult-bounds/tree/463113d675af7410ac91d53a4fed51ec6494eb9a/research/selected-bit-recursive-closure.
[10] research/selected-bit-stream-routes, in [1] at 463113d6,
https://github.com/CrocSwap/integer-mult-bounds/tree/463113d675af7410ac91d53a4fed51ec6494eb9a/research/selected-bit-stream-routes.
[11] research/e8-bit-p111-tuned/code/adapt9p.py, in [1] at 0fdd7d82,
https://github.com/CrocSwap/integer-mult-bounds/blob/0fdd7d82de03e43c9e5a1000dfbcd380d1b687d2/research/e8-bit-p111-tuned/code/adapt9p.py.

Email address: jdb1729@gmail.com

Email address: platypii@gmail.com